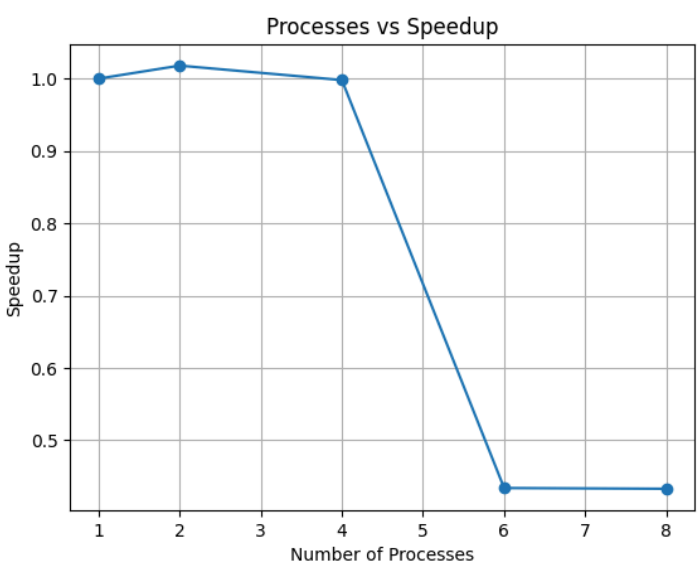
ASSIGNMENT-4

GAURANG GARG [102303134]

QUESTION-1:

```
Compiling q1.cpp ...
Compilation successful!
----------------------------------------
Running with 1 process(es)...
Execution Time (Tp) with 1 processes: 0.202020 seconds
----------------------------------------
Running with 2 process(es)...
Execution Time (Tp) with 2 processes: 0.198429 seconds
----------------------------------------
Running with 4 process(es)...
Execution Time (Tp) with 4 processes: 0.202493 seconds
----------------------------------------
Running with 6 process(es)...
Execution Time (Tp) with 6 processes: 0.465973 seconds
----------------------------------------
Running with 8 process(es)...
Execution Time (Tp) with 8 processes: 0.466860 seconds
----------------------------------------
```

Processes (p)	Tp (s)	Speedup (S)	Efficiency (%)	Communication (%)
1	0.202020	1.000	100.00%	0.00%
2	0.198429	1.018	50.90%	49.10%
4	0.202493	0.998	24.94%	75.06%
6	0.465973	0.434	7.23%	92.77%
8	0.466860	0.433	5.41%	94.59%



Observations:

No real speedup was expected here - and the results confirm it. Every process runs the exact same 200M iteration loop regardless of how many processes are launched, so adding more processes never reduces anyone's workload.

p=2 showing a slightly faster time than p=1 is just timing noise or cache warmup - not a genuine parallel gain.From p=1 to p=4 performance stays almost flat, meaning MPI communication cost is negligible at this scale. The machine likely has 4 physical cores, so each process gets its own core with no contention.

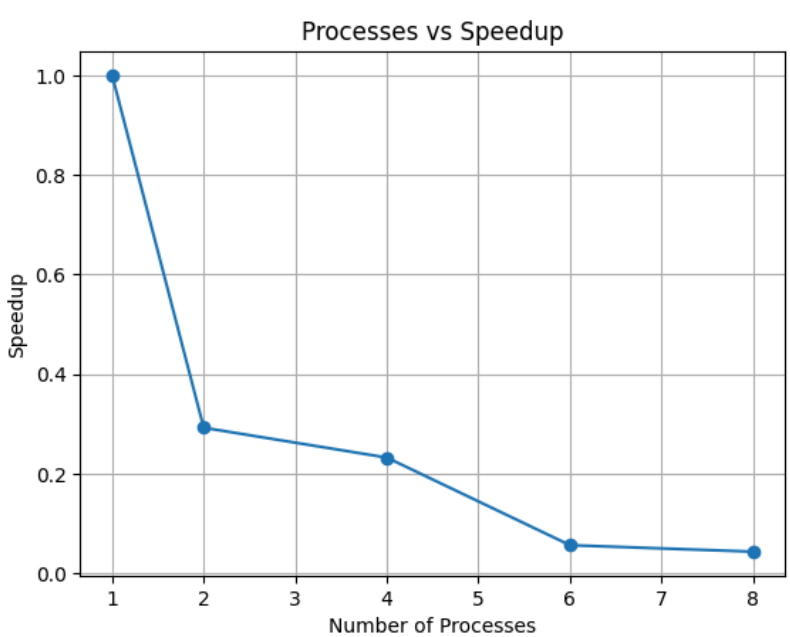
At p=6 and p=8 execution time more than doubles. This is the machine running out of physical cores - processes start competing for CPU time, causing context switching on top of the growing ring communication overhead.

Efficiency collapses from ~51% at p=2 down to just ~5.4% at p=8. Since there's no workload decomposition happening at all, this degradation was entirely predictable - more processes just means more overhead with zero computational benefit.

QUESTION-2:

```
Compiling q2.cpp ...
Compilation successful!
----------------------------------------
Running with 1 process(es)...
Computed Sum: 5050
Execution Time (Tp): 0.00000281 seconds
----------------------------------------
Running with 2 process(es)...
Computed Sum: 5050
Execution Time (Tp): 0.00000962 seconds
----------------------------------------
Running with 4 process(es)...
Computed Sum: 5050
Execution Time (Tp): 0.00001212 seconds
----------------------------------------
Running with 6 process(es)...
Computed Sum: 5050
Execution Time (Tp): 0.00005051 seconds
----------------------------------------
Running with 8 process(es)...
Computed Sum: 5050
Execution Time (Tp): 0.00006554 seconds
----------------------------------------
```

Processes (p)	Tp (s)	Speedup (S)	Efficiency (%)	Communication (%)
1	0.00000281	1.000	100.00%	0.00%
2	0.00000962	0.292	14.60%	85.40%
4	0.00001212	0.232	5.79%	94.21%
6	0.00005051	0.056	0.93%	99.07%
8	0.00006554	0.043	0.54%	99.46%



Observations :

The computed sum is consistently **5050** across all runs, which is correct (sum of 1 to 100), so the program is logically correct throughout.

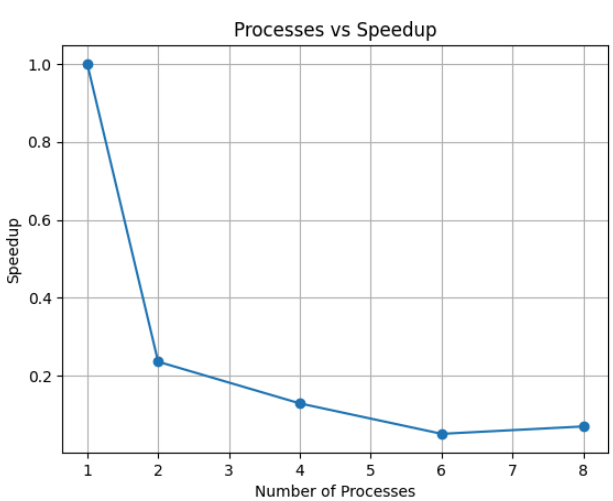
However, performance tells a different story. Adding more processes actually **slows things down** rather than speeding them up. The task - summing 100 integers - is simply too small to benefit from parallelism. By p=8, over **99% of the time is pure communication overhead**, meaning the processes spend almost all their time talking to each other rather than doing actual work.

This is a classic case of **over-parallelizing a lightweight task**. The MPI calls (Scatterv + Reduce) cost far more time than the computation itself. A single process finishes in ~0.28 microseconds, but 8 processes together take ~6.5 microseconds - nearly **23x slower**. The sweet spot here is clearly p=1, and scaling up only hurts.

QUESTION-3:

```
Compiling q3.cpp ...
Compilation successful!
----------------------------------------
Running with 1 process(es)...
Global Maximum: 970 (Process 0)
Global Minimum: 399 (Process 0)
Execution Time (Tp): 0.00000172 seconds
----------------------------------------
Running with 2 process(es)...
Global Maximum: 997 (Process 1)
Global Minimum: 197 (Process 1)
Execution Time (Tp): 0.00000728 seconds
----------------------------------------
Running with 4 process(es)...
Global Maximum: 997 (Process 0)
Global Minimum: 9 (Process 2)
Execution Time (Tp): 0.00001337 seconds
----------------------------------------
Running with 6 process(es)...
Global Maximum: 997 (Process 0)
Global Minimum: 9 (Process 2)
Execution Time (Tp): 0.00003386 seconds
----------------------------------------
Running with 8 process(es)...
Global Maximum: 995 (Process 0)
Global Minimum: 9 (Process 1)
Execution Time (Tp): 0.00002440 seconds
----------------------------------------
```

Processes (p)	Tp (s)	Speedup (S)	Efficiency (%)	Communication (%)
1	0.00000172	1.000	100.00%	0.00%
2	0.00000728	0.236	11.81%	88.19%
4	0.00001337	0.129	3.22%	96.78%
6	0.00003386	0.051	0.85%	99.15%
8	0.00002440	0.070	0.88%	99.12%



Observations:

The program correctly finds the global min and max across all processes every time, and also identifies which process owned that value - a nice touch using MPI_MAXLOC and MPI_MINLOC.

But just like q2, the actual computation here is extremely lightweight - just finding min/max across 10 random numbers per process. That's done in microseconds, so MPI communication ends up being the dominant cost by a wide margin. By p=4, nearly 97% of the time is overhead, not actual work.

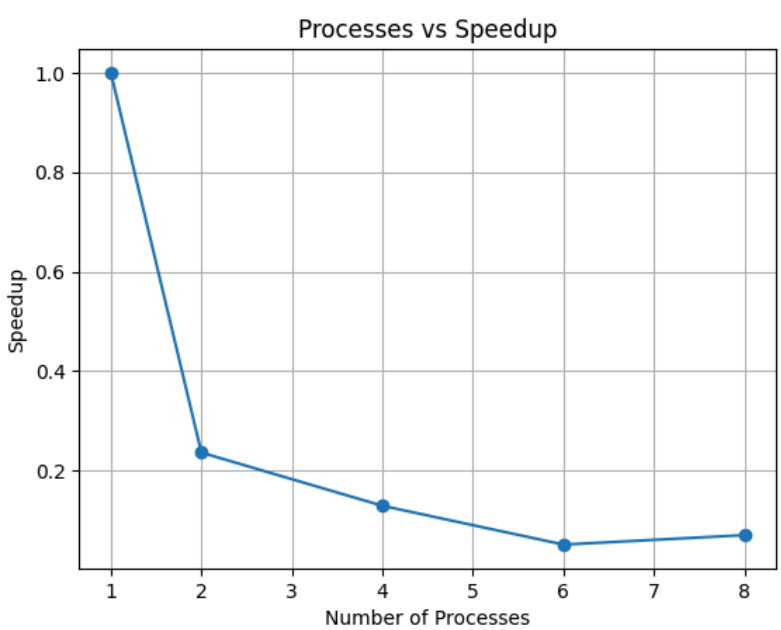
One interesting thing - p=8 is slightly faster than p=6 (2.44µs vs 3.39µs). This is just timing noise; at such tiny timescales, results fluctuate based on system load, CPU scheduling, and MPI internals rather than any real algorithmic difference.

The bottom line is the same as before: when the task is this small, parallelism hurts more than it helps. This code would only start showing real gains if each process handled thousands or millions of values instead of just 10.

QUESTION-4:

```
Compiling q4.cpp ...
Compilation successful!
---------------------------------------
Running with 1 process(es)...
Dot Product: 120
Execution Time (Tp): 0.00000233 seconds
---------------------------------------
Running with 2 process(es)...
Dot Product: 120
Execution Time (Tp): 0.00001109 seconds
---------------------------------------
Running with 4 process(es)...
Dot Product: 120
Execution Time (Tp): 0.00002594 seconds
---------------------------------------
Running with 6 process(es)...
Vector size must be divisible by number of pro
---------------------------------------
Running with 8 process(es)...
Dot Product: 120
Execution Time (Tp): 0.00003481 seconds
---------------------------------------
```

Processes (p)	Tp (s)	Speedup (S)	Efficiency (%)	Communication (%)
1	0.00000233	1.000	100.00%	0.00%
2	0.00001109	0.210	10.50%	89.50%
4	0.00002594	0.090	2.25%	97.75%
6	-	-	-	-
8	0.00003481	0.067	0.84%	99.16%



Observations :

The dot product result is consistently 120 across all valid runs, which checks out - vectors [1-8] and [8-1] dotted together give exactly 120, so correctness is confirmed.

Performance-wise, this is the same story again. Computing a dot product of just 8 elements is an incredibly tiny task, and MPI communication overhead completely drowns it out. By p=8, 99% of the time is pure overhead with almost no time spent on actual multiplication.

The p=6 case is worth noting - the program correctly handles the edge case where the vector size isn't evenly divisible, exiting gracefully instead of producing wrong results. That's good defensive programming.

Overall, parallel dot product only makes sense at much larger vector sizes (think millions of elements). At 8 elements, even p=2 is nearly 5x slower than just doing it sequentially.